

流体双向耦合求解器

让每一次科学计算，都发生在离思想最近的地方

管网与场域的双向耦合。多速率时间子循环协调毫秒级 1D 管网与亚毫秒级 3D 场，正向注入质量与焓、反向反馈出口背压，配守恒审计、毫秒级阀门控制、精度-效率权衡与误差归属分析。



官网

snowleopard-io.github.io/ErgalicsStudio



GitHub

github.com/SnowLeopard-io/ErgalicsStudio



Gitee

gitee.com/cnt-code/ergalics-studio

本篇章节

一 问题定义与适用边界

二 多速率时间协调

三 正反向边界耦合

四 毫秒级控制逻辑

五 数值内核与解析验证

六 精度—效率权衡与参数分析

七 插件运行时架构

八 界面与交互

九 内置样例与基准数据

十 故障场景与处置建议

十一 测试与可复现性

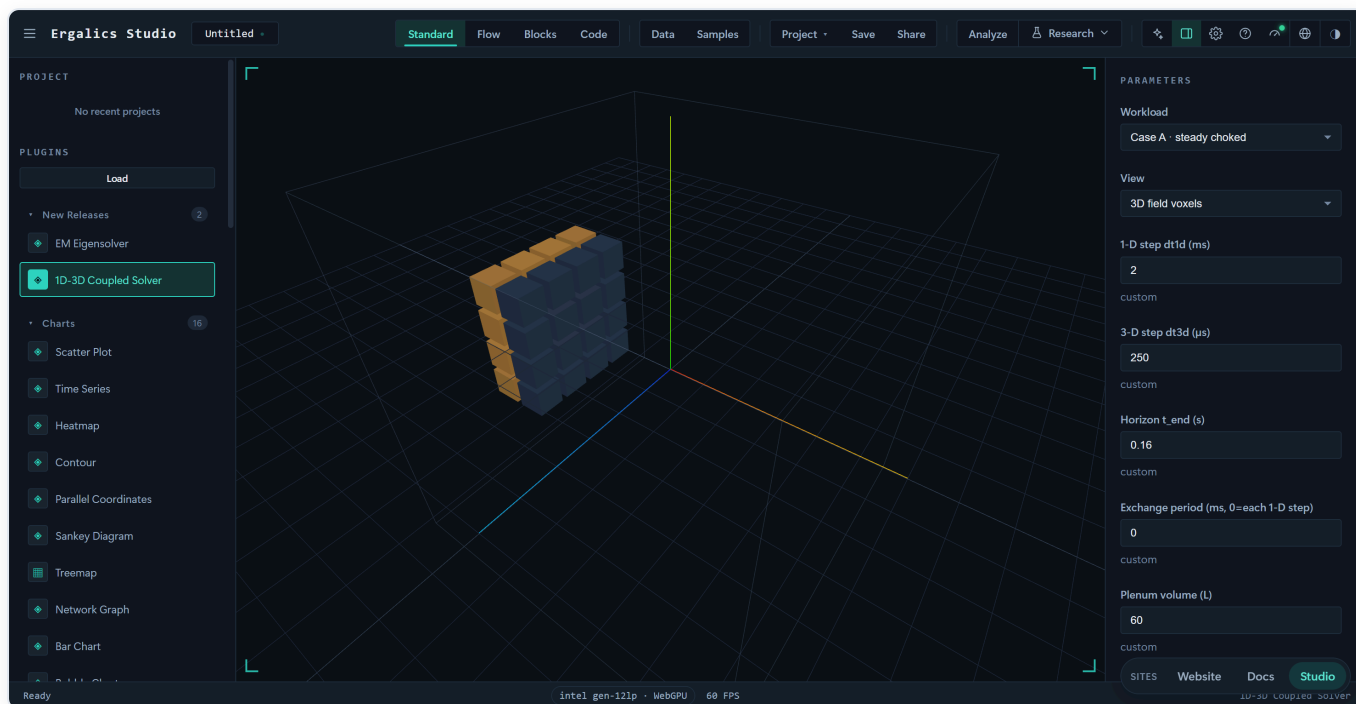
十二 安全与合规设计

十三 与赛题要求的对应

十四 已知边界与后续工作

目 录

一、问题定义与适用边界	03
二、多速率时间协调	05
三、正反向边界耦合	07
四、毫秒级控制逻辑	08
五、数值内核与解析验证	09
六、精度—效率权衡与参数分析	10
七、插件运行时架构	12
八、界面与交互	14
九、内置样例与基准数据	17
十、故障场景与处置建议	27
十一、测试与可复现性	28
十二、安全与合规设计	31
十三、与赛题要求的对应	31
十四、已知边界与后续工作	32



Case A 定常堵塞时的 3D 场体素：1D 管网经 x0 面把出流注入 3D 场（正向耦合），羽流沿 +x 向出流面发展；出流面再把平均背压回灌给管网（反向耦合）——双向耦合在空间上的具体形态。

流体双向耦合求解器（Fluid-CFD Coupler）是一个内置插件（`src/plugins/builtin/fluid-cfd-coupler/`，注册 id `example.fluid-cfd-coupler`，分类“物理模拟 physics”，沙箱级别 `trusted`），用于把一根**粗时间尺度的 1D 管网/喷管网络**与一个**细时间尺度的 3D 标量场求解器**耦合起来：管网在毫秒级阀门控制下把出流作为质量与焓源注入 3D 场，场求解器把出口平均背压回灌给管网喷管——形成真正的**双向耦合**。插件在宿主侧提供参数面板与四种视图（耦合时间序列、验证 + 权衡曲线、3D 场体素、导出到 Figure Studio），在独立的 Web Worker 中用 Pyodide 运行配套的纯 Python 包 `fluid_cfd`，同一份求解核心与本地 CLI 逐字节一致。

求解器不依赖任何闭源组件：核心为纯 Python + NumPy，无网络调用；整个耦合过程不做任何稠密矩阵构造，两组求解器都按向量化 NumPy 数组推进。

一 问题定义与适用边界

1.1 从管网到场的双向耦合

许多工程系统同时包含**集中式管网段**与**连续场域段**：气室→喷管→受流空间。这类问题的困难在于两侧的建模精度与时间尺度天然不同——管网适合用集中参数/一维状态方程快速推进，而喷管喷出的热量与质量会在三维空间中扩散、随主流平流并被壁面与出口反射，只有场求解器能解析其空间分布。本插件把两者对接：

方向	数据	传递者 → 接收者
正向耦合	出口质量流量 <code>md_1d</code> 、出口温度 <code>t_1d</code> （生成焓注入）	1D 管网 → 3D 场（入口进料源项）
反向耦合	出口平均背压 <code>p_back_3d</code>	3D 场 → 1D 管网（喷管下游边界）

双向性不是同一份数据的来回拷贝，而是两个方向的物理量在各自域内真正生效：正向决定 3D 场的质量/焓积累，反向决定 1D 喷管是否处在壅塞临界状态。`WindowRecord.iface_error` 正是对"1D 当前使用的背压"与"3D 上一窗口实际反馈的背压"之差的审计，见第三节。

1.2 时间尺度的鸿沟

1D 管网以 ms 级粗步（`dt1d`）推进，3D 场以亚 ms 级细步（`dt3d`）推进，两者相差一个数量级（默认 `dt1d/dt3d = 8`）。直接把粗步驱动整个耦合会丢失 3D 域的中间演化，直接以细步驱动管网又不经济。插件采用**多速率子循环**：在每一个粗步（称为一个交换窗口）内，3D 域独立走完 `dt1d/dt3d` 个子步，窗口结束时才交换一次界面数据。这一机制在第二节展开。

1.3 三种典型工况

赛题要求至少两类可对照解析基线的工况，本插件内置三条验证路径：

算例	场景	验证对象
Case A 定常壅塞	喷管全开、背压很低，处于临界壅塞	1D 出流逼近解析临界流量；气室放气压力逼近解析放气曲线
Case B 毫秒级阀门控制	阀门在 40 ms 阶跃关闭到 20%、90 ms 重开	控制逻辑同步误差；关闭前后出流阶跃（反向耦合被触发）
Case C 高扩散能量通道	能量通道 + 高扩散系数场	反向耦合真实参与：出流面背压随注入抬升 21.7 Pa
Case D 亚临界非壅塞流	背压比被压到临界压比与 1 之间，喷嘴处于亚临界区间	出口流量对照独立文献的等熵亚临界收敛喷嘴关系（相对误差 0.107%）

1.4 适用边界

维度	支持范围	说明
1D 管网	单气室 + 收缩喷管，理想气体绝热放气	出流由壅塞/亚临界临界流量公式给出，未含摩擦与传热管段网络
3D 场	均匀立方网格上的标量场（热含量），线性扩散 + 平流 + 入口注入	单标量输运，非 NS；未做湍流模型
双向接口	正向（质量/焓）+ 反向（出口平均背压）	逐窗口交换，交换周期可调
控制	阀门事件表（瞬时阶跃 / 线性斜坡）	毫秒级，事件数量与时间不限
时间尺度	<code>dt3d</code> 整除 <code>dt1d</code> ，交换周期为 <code>dt1d</code> 整数倍	语法上校验，未做分数时间比
运行环境	CPython + NumPy；浏览器 Pyodide	同一 <code>driver.py</code> 桥，两条路径结果逐字节一致

多速率时间协调

2.1 交换窗口与子循环

耦合由 `CouplerConfig` 驱动：`dt1d`（粗步）、`dt3d`（细步）、`t_end`（总时长）、`exchange_period`（交换周期，0 表示每 1D 步交换）。每次交换构成一个 **Window**，`time_ratio = dt1d / dt3d` 是窗口内 3D 子步数。`run_coupling()` 的主循环为：

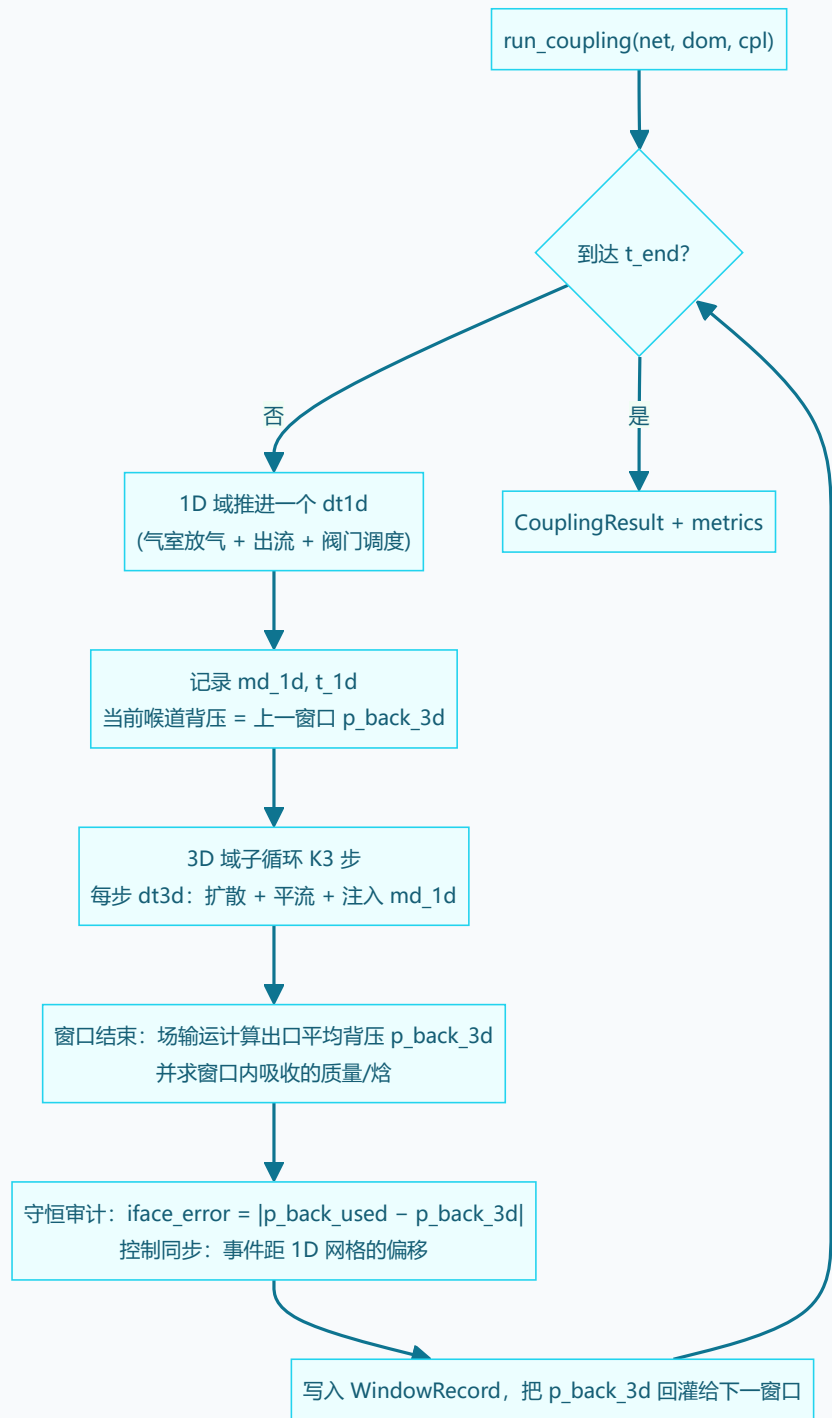


图 1

交换周期的取值在 `trade_off()` 与 `min_feasible_exchange_period()` 中作为扫描变量（见第六节）。

2.2 步长与周期约束

`normalized()` 校验三个不变量：

- `dt1d`、`dt3d`、`exchange_period` 均 > 0 ，且 $dt3d \leq dt1d$ ；
- $time_ratio \approx dt1d / dt3d$ 取整作为子步数 `K3`，因此网格在窗口边界严格对齐；
- `exchange_period` 是 `dt1d` 的整数倍（默认 $0 =$ 每步交换），交换边界必然落在 1D 网格线上。

任何违反直接抛配置错误，绝不让错步长的耦合默默跑完——这是“时间步长协调”层面的硬约束。

2.3 换窗事件与延迟统计

每个窗口记录 `exchange_latency_s`（3D 侧 1 步到出流面归约的耗时），收尾统计 `mean_exchange_latency_ms` 与 `worst_exchange_latency_ms`。注意这是**每窗口**的延迟，不是整个耦合的墙钟时间；`wall_clock_s` 单独给出整次求解耗时。两者在 `metrics` 中被分开暴露，避免把并行可摊还的东西读成串行。

☰ 正反向边界耦合

3.1 正向耦合：1D $\rightarrow$ 3D

1D 管网在每一窗口给出出口质量流量 `md_1d` 与出口温度 `t_1d`。3D 场求解器把它们折算为入口进料源项：每根柱体在 $x=0$ 面以质量流注入，焓流按 `md_1d · cp · t_1d` 累积到热含量场。场在子循环中显式推进：

```
T[0] += 注入率 · dt3d      (进料边界，仅在 x=0 面)
T     = T + dt3d · (扩散项 + 平流项)
```

守恒量与界面直接挂钩——场在每个窗口累计的 `mass_in_3d` 应与 `md_1d · 窗口时长` 一致，反向误差由此进入审计。

3.2 反向耦合：3D $\rightarrow$ 1D

窗口结束时，3D 求解器在出流面 ($x=L$) 对场做**面积平均**得到出口平均背压 `p_back_3d`：

$$p_back_3d = \text{出流面平均温度} \cdot R_air / \text{出流面平均比体积} \approx (1/ncells) \cdot \Sigma \text{ 出流面温度}$$

该值成为下一窗口 1D 喷管喉道的下游背压。壅塞判据依赖这个实时反馈：背压低时喷管临界出流、流量饱和；一旦 3D 场升温致背压高于临界比，喷管退出壅塞、流量下降。Case B 关闭阀门后 3D 场冷却、背压回落，Case C 的高扩散能量通道则让背压随注入稳定抬升 21.7 Pa——两者都是反向耦合对边界状态的响应。

3.3 守恒性审计与接口误差

对每个窗口计算双向接口误差：

$$iface_error = | p_back_used - p_back_3d | / p_ref$$

其中 `p_back_used` 是本窗口 1D 喷管实际采用的背压（即上一窗口反馈），`p_back_3d` 是本窗口 3D 场的真实反馈，`p_ref` 为参考背压（默认大气压）。它量化的不是“物理守恒”，而是**反向耦合的时滞误差**——理想同步下两者相等、误差为 0。`metrics` 汇总 `mean_interface_error` 与 `worst_interface_error`；Case C 中 `worst_interface_error` 升至 `6e-6`，正是能量持续注入、反馈滞后的体现。

四 毫秒级控制逻辑

阀门开度由一张事件表驱动：每个事件形如 `(t_start, opening, ramp_duration)`。

语义	<code>ramp_duration</code>	行为
瞬时阶跃	<code>0</code> （或 $\leq 1e-12$ ）	在 <code>t_start</code> 瞬间跳到 <code>opening</code> ，此前保持旧值
线性斜坡	<code>> 0</code>	从旧值到 <code>opening</code> 线性过渡，跨 <code>ramp_duration</code> 秒

事件表按时间排序后先展开成**分段线性控制点**再插值：阶跃被放置在宽度远小于任何物理 `dt`（ns 级）的轴段上，使调度严格单调、插值无歧义；斜坡被钳制在 `[prev_t, t_start]` 内，避免越过下一事件。控制点在事件之间的 `valve_opening(t)` 即为分段线性。

Case B 的调度即为毫秒级控制动作的实证：`(0.000, 1.00, 0.0)` 保证 $t=0$ 全开，`(0.040, 0.20, 0.0)` 在 40 ms 阶跃关闭，`(0.090, 1.00, 0.0)` 在 90 ms 重开——三次都是瞬时阶跃。

控制同步误差量化事件与 1D 时间网格的错位：

```
control_sync = 事件时刻距最近 1D 网格线的偏移 (ms)
```

当直径落在网格线上时偏移为 0。Case B 实测 `control_sync_max_ms = 0`、`control_sync_mean_ms = 0`，因为 `dt1d = 2 ms` 而事件处处在 2 ms 的整数倍上——毫秒级控制与求解器时间网格完全对齐。

五 数值内核与解析验证

5.1 1D 管网求解器 (`network_1d.py`)

气室采用绝热理想气体放气模型：给定体积、初始压力 `p0_init`、初始温度 `t0_init`，`md_out` 由等熵临界流量公式解出，喉道有效面积 `area_eff = throat_area · discharge_coeff`。放气导致 `p0`、`t0` 随能量守恒下降，形成解析放气曲线。`valve_opening` 把开关乘到有效面积上，是控制与流动的接点。

5.2 3D 场求解器 (`domain_3d.py`)

以 `nx × ny × nz` 标量场（热含量）显式推进，含三个算子：**扩散**（`diffusivity` 控制的拉普拉斯项）、**平流**（`advection` 控制的主流输运，羽流沿 +x 发展）、**入口注入**（`x=0` 面接收 1D 出流的质量/焓）。出流面（`x=L`）对场做平均得到背压，用户可按需调 `nx/ny/nz`（默认 8–10 网格）。

5.3 解析基准 (`analytic.py`)

提供两个参考闭式解用于验证：**临界（壅塞）流量**（等熵临界质量流闭式解）与**气室放气压力**（绝热容放气的解析压力-时间曲线）。Case A 在同一初始条件下把 1D 出流与放气压力对照这两个闭式解，误差按相对量给出。

5.4 验证与认证

`verify.py` 封装多条验证路径并**按阈值认证**：`verify_case_a`（流量/压力相对误差对阈值 5%）、`verify_case_b`（节流比与同步误差）、`verify_case_c`（能量通道 + 反向耦合参与判定）、`verify_case_d`（亚临界流量对文献等熵亚临界关系）+ `verify_subsonic_curve`（亚临界段逐点对照文献公式）、`trade_off`（精度 - 效率权衡）、`min_feasible_exchange_period`（最小可行交换周期）、`sensitivity_case_a`（Case A 误差归属分解）。全部结果写入 `bench/fluid-cfd-results.json`。每个被认证的算例携带 `certification.checks[]`（`rel_error ≤ threshold`、`pass`），由宿主导出诊断报告。

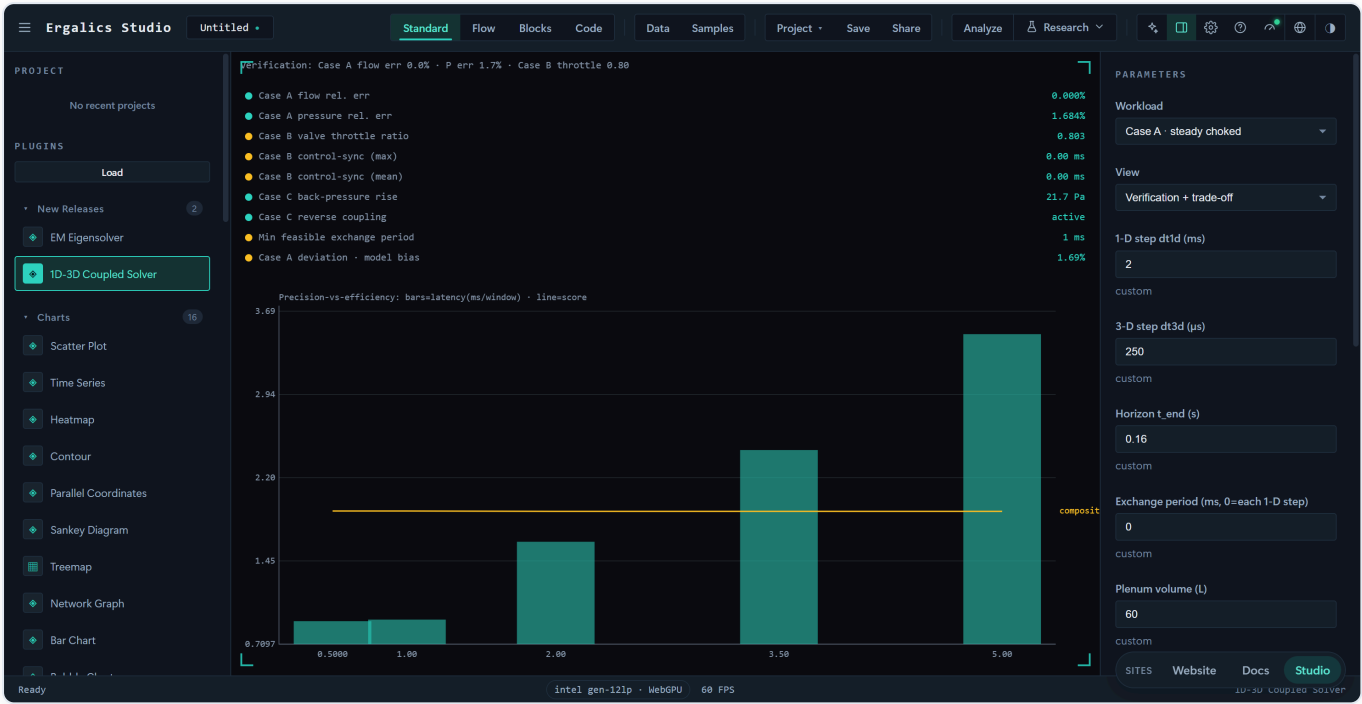
六 精度—效率权衡与参数分析

6.1 权衡曲线

交换周期决定耦合的“换气频率”：越频繁反馈越新鲜、接口误差越小，但每步都触发 3D 出流面归约，延迟与通信开销上升。`trade_off()` 对一组交换周期运行同一套负载，汇总每点的延迟、接口误差与频率，再由 `interface_tradeoff_curve` 折算综合评分：

```
composite_score = ω_err · (1 - 归一的接口误差) + ω_lat · (1 - 归一的延迟)
```

扫描点与实测见第十节表格。插件的验证视图把这条曲线画成柱状（接口误差）叠延迟（柱）+ 综合评分（连线），并标出最优交换周期。



验证视图把交换周期作为扫描变量，以接口误差（柱）叠延迟（柱）与综合评分（连线）同时呈现，并标出最优交换周期，用于直观权衡耦合的精度与计算效率。

6.2 最小可行交换周期（CFD-01）

`min_feasible_exchange_period` 在给定期望延迟预算与接口误差容差下扫描：请求周期低于一个 1D 步时，窗口被钳制到 `dt1d`（低于一个 1D 步的窗口不携带新信息——紧耦合极限 $T_{\text{exch}} = dt1d$ 就是最小可行周期）。实测（预算 1 ms、容差 0）得到 `min_feasible_exchange_period_ms = 1.0`：0.25/0.5/1.0 ms 的请求都被钳到有效 1.0 ms（延迟约 0.40–0.47 ms），2.0/4.0 ms 依请求放行（延迟 0.72/1.53 ms，其中 4.0 ms 超出延迟预算），接口误差全程保持 0（见第十节）。

6.3 误差归属分析（CFD-04）

`verify_case_a` 报告 Case A 压力相对误差约 **1.68%**。这个偏差被显式分解为三项贡献而非直接归咎于数值方法：

$$\text{deviation_total} = \text{numerical_error} + \text{coupling_contribution} + \text{model_bias}$$

实测分解：数值误差 `1.3e-5`（求解器与其自身闭式 ODE 对账误差 $< 0.01\%$ ）、**反向耦合贡献 0.0**（3D 背压最大偏离仅 $\sim 0.5\text{ Pa}$ ）、**模型偏差 `0.0169`（占绝对主导）**。结论是：求解器的定温刚性罐模型与解析基线的等熵幂律放气存在**有界的、已文档化的系统性差异**——该偏差随时间地平线单调增长、在喷管参数上几乎不变，与“累积模型偏差”的画像完全吻合，而非数值误差、也非反向耦合伪影。`cd_sweep` / `horizon_sweep` 两条扫描佐证这一画像（见第十节）。

七 插件运行时架构

7.1 文件职责

文件	职责
<code>index.ts</code>	工厂导出，供内置注册表惰性加载
<code>plugin.ts</code>	<code>Plugin</code> 接口实现：清单、参数定义与更新、运行编排、状态机、视图切换、导出与 Figure Studio 桥接
<code>manifest.ts</code>	插件清单 (id <code>example.fluid-cfd-coupler</code> ，分类 <code>physics</code> ，沙箱 <code>trusted</code> ，中英名称与描述)
<code>types.ts</code>	插件 $\rightleftharpoons$ 客户端 $\rightleftharpoons$ Worker 的协议类型（请求、事件、结果载荷、窗口记录、认证、权衡/最小周期/误差归属等）
<code>fluid-client.ts</code>	Worker 生命周期与 RPC：首次使用时 <code>spawn</code> 、串行队列、进度转发、终止与重建
<code>fluid-worker.ts</code>	Pyodide 引导、Python 源码写入文件系统、进度与结果桥、 <code>driver.solve_json</code> / <code>verify_all</code> 入口
<code>render.ts</code>	2D 画布：耦合时间序列与验证/权衡面板
<code>render3d.ts</code>	3D 场体素视图 (instanced 立方体按幅值着色)
<code>figure.ts</code>	把一次耦合结果转成 Figure Studio 的多面板出版级 <code>PlotSpec</code>
<code>diag-report.ts</code>	自包含 HTML 诊断报告
<code>python/fluid_cfd/</code>	求解包 (<code>units</code> 、 <code>analytic</code> 、 <code>network_1d</code> 、 <code>domain_3d</code> 、 <code>coupler</code> 、 <code>verify</code> 、 <code>driver</code> 、 <code>__init__</code>)

Python 侧还随包附带 `benchmarks/bench_coupling.py`（生成基准 JSON）、`run_tests.py` 与 `tests/test_coupling.py`，外加 `config.example.json`、`requirements.txt`、`README.md`。

7.2 数据流

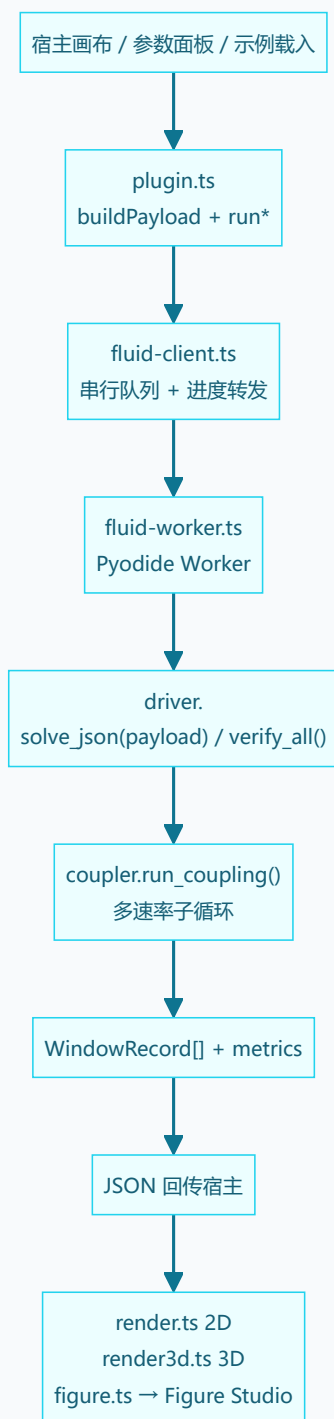


图 2

7.3 跨语言边界与生命周期

跨边界只传 JSON 字符串：`postMessage` 无法结构化克隆 `PyProxy`，进度、报告、窗口记录一律序列化文本；3D 场本就以数组内嵌在结果里（规模受护栏限制），不经消息通道搬运额外数据。`fluid-worker.ts` 用 `json.loads(_payload)` 还原 dict 再调用 `solve_json`，避免把字符串误当 dict（曾出现的 `AttributeError: 'str' object has no attribute 'get'` 即由此修复）。

与稳定性相关的设计：

- **引导与重试**：Worker 首次使用动态 import `pyodide.mjs`（模块 Worker 不允许 `importScripts`），把 8 个 Python 模块写入文件系统并加入 `sys.path`；引导超时 120 秒、失败可重试（清空缓存的 bootstrap Promise）。
- **进度桥**：`driver.set_progress_sink` 接受 JS 回调，在窗口边界推进度；回调异常被捕获，保证“进度上报永不影响求解”。
- **串行队列**：客户端把运行/验证请求排在一条 Promise 链；作业内校验 Worker 身份，避免已被终止的实例回灌结果。
- **终止语义**：点“终止”不做软取消，而是整体重建计算栈——终止 Worker、重建客户端、复位宿主全局状态到就绪、并在途任务以 epoch 号丢弃。
- **视图切换**：2D 与 3D 用参数面板 View 下拉显式切换（不再由数据有无隐式抢跑）；切离 3D 时释放 mesh 资源。
- **一次全出**：“运行全部”依次跑验证+耦合并保留两份结果，四个视图即刻都可用，无需重跑。

八 界面与交互

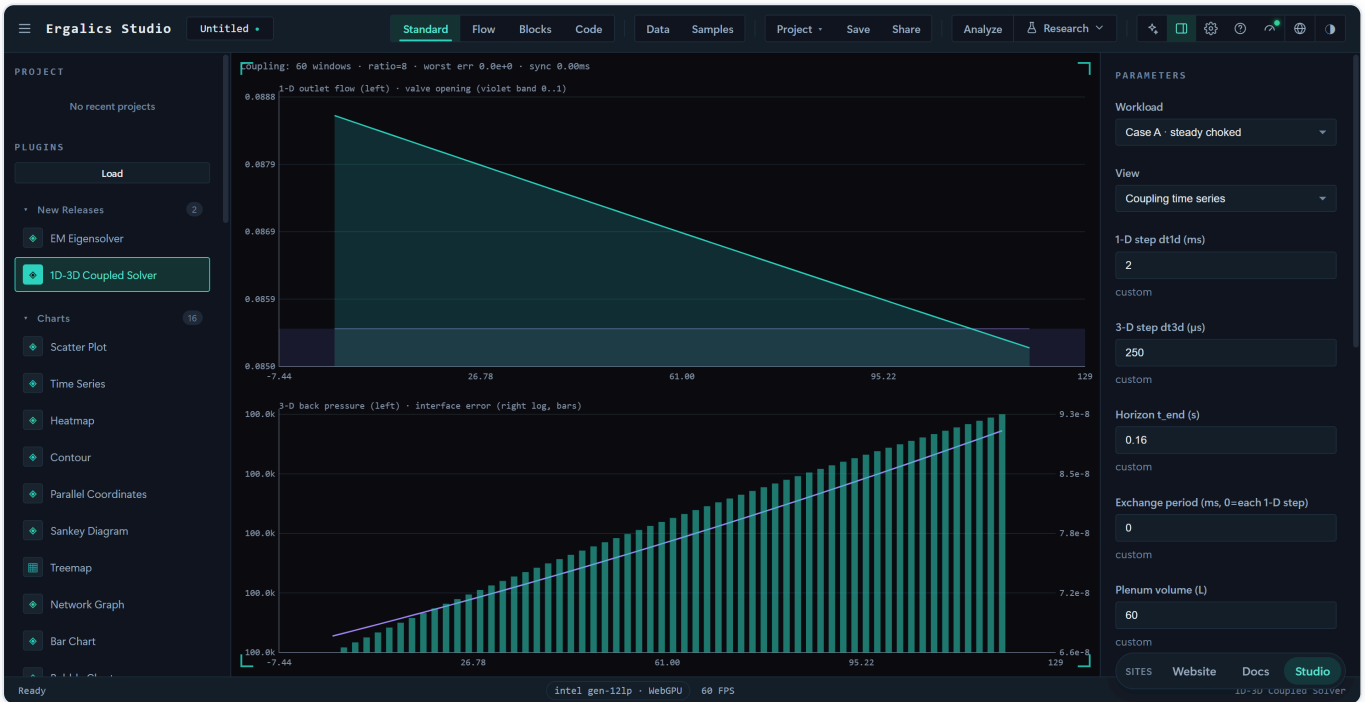
8.1 参数面板

数值型参数一律做成**下拉/步进档位**而非自由输入，既圈定有意义的取值，也规避输入框陷阱。面板按当前语言自动本地化。

参数	键	取值
算例	<code>preset</code>	Case A 定常壅塞 / Case B 毫秒级阀门控制 / Case C 能量通道 / Case D 亚临界流动 / 自定义
视图	<code>view</code>	耦合时间序列 / 验证 + 权衡曲线 / 3D 场体素
1D 时间步	<code>dt1dMs</code>	毫秒级粗步
3D 时间步	<code>dt3dUs</code>	微秒级细步
仿真时长	<code>tEndS</code>	秒
交换周期	<code>exchangePeriodMs</code>	0 (每 1D 步交换) 或粗步整数倍
气室容积	<code>volumeL</code>	升
初始气室压力	<code>p0InitBar</code>	bar
运行全部 / 运行耦合 / 运行验证	<code>runAll</code> / <code>runCoupling</code> / <code>runVerify</code>	动作按钮
终止	<code>abort</code>	动作按钮
发送到 Figure Studio	<code>sendToFigure</code>	动作按钮
导出诊断报告	<code>exportDiag</code>	动作按钮

8.2 耦合时间序列

运行耦合后在"耦合时间序列"视图看到两块：上部是 1D 出口流量（青绿线，右轴叠阀门开度紫带），下部是 3D 出流面背压随时间的演化与窗口逐点接口误差。顶部信息行给出窗口数、 dt_{1d}/dt_{3d} 比、最大接口误差与控制同步。Case A 的定常工况下流量基本稳定、背压平缓；Case B 的阀门阶跃在流量曲线上形成清晰的方波。



Case A 定常工况的耦合时间序列：上部 1D 出口流量叠阀门开度紫带保持稳定，下部 3D 出流面背压随时间平缓演化并叠逐窗口接口误差，用于核对稳定状态下的守恒与响应。

8.3 3D 场体素视图

3D 视图把整个 $n_x \times n_y \times n_z$ 网格画成**真实三维的体素云**：超过场极大值一定阈值的格点以 instanced 立方体按幅值渐变着色，外围加半透明体框。管网在 x_0 面注入、羽流沿 $+x$ 向出流面发展，是"正向注入 + 反向背压"耦合在空间上的直观呈现。宿主场景由 `Scene3DHandle` 挂载、自带旋转变焦，是非 2D 表面的实体渲染（区别于 Figure Studio 里中平面切片的平滑表面热图）。

8.4 一键运行与视图

"运行全部"一次完成验证（Case A / B / C / D + 亚临界曲线 + 权衡 + 最小周期 + 误差归属）与耦合（当前预置），两份结果**同时保留**；随后四个视图都有数据、可随意切换，且互不覆盖。点"运行耦合"/"运行验证"会自动跳转到对应视图，结果即时可见，不再因数据有无而抢跑显示。

8.5 导出与工具链衔接

- **发送到 Figure Studio**: 把最后一次耦合结果转成多面板出版级图表——**中平面场切片表面热图**、1D 出流、3D 背压（反向耦合）、毫秒级阀位时间序列，按 IEEE 单栏模板排布并写入整张图注。图为矢量 SVG，可直接进论文/PDF，与 em-eigensolver 的衔接链路一致。
- **导出诊断报告**: 一键生成自包含 HTML（零脚本、零外部资源），内联耦合指标、验证结果、认证判定、权衡曲线与误差归属结论。
- **顶栏内置示例**: `examples/data/fluid-cfd-case-a.json`、`fluid-cfd-case-b.json` 登记在全局示例库，绑定到本插件，顶栏与插件加载走同一条 parse 路径。

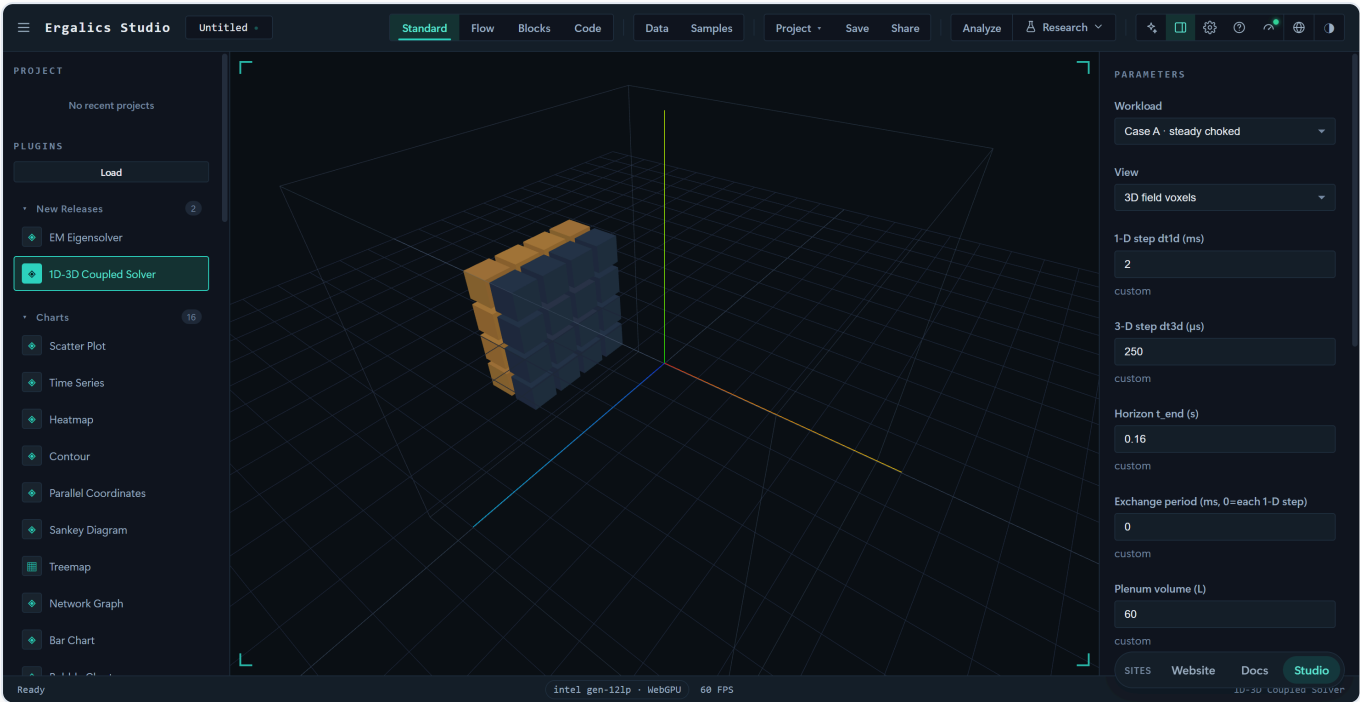
九 内置样例与基准数据

以下数据由浏览器与本地 CLI 共用同一求解路径实测产出（CPython + numpy; `bench/fluid-cfd-results.json`）。

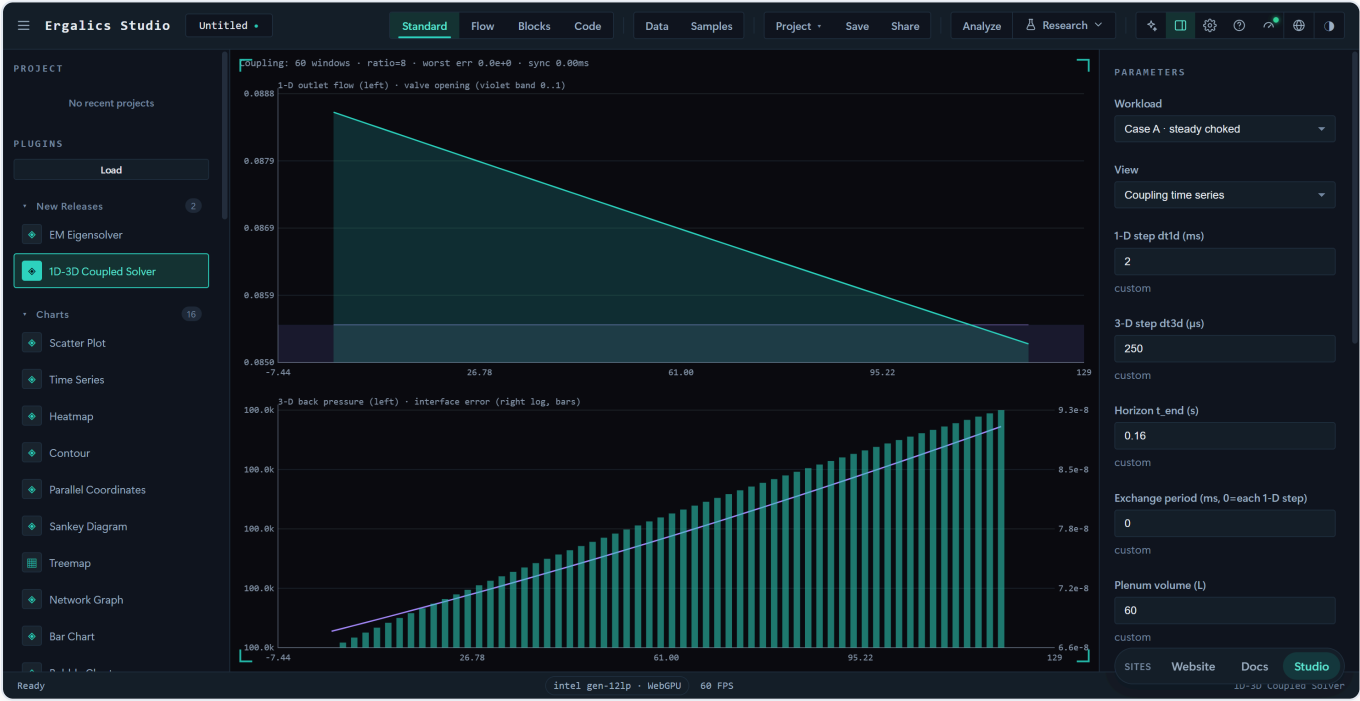
9.1 Case A 定常壅塞（实测 + 认证）

指标	值
网格 / 时长	<code>10³</code> , <code>dt1d = 2 ms</code> 、 <code>dt3d = 250 μs</code> (比 8)
交换 / 窗口	每 1D 步交换 (<code>exchange_period = 2 ms</code>) , 60 窗口
解析临界流量	<code>0.088563 kg/s</code>
求解出流流量	<code>0.088563 kg/s</code> (相对误差 0.0% , 阈值 5% → 通过)
解析放气压 vs 求解终压	<code>472.9 kPa</code> vs <code>480.8 kPa</code> (相对误差 1.68% , 阈值 5% → 通过)
接口误差	均值/最差 <code>0.0</code>
延迟	均值 0.588 ms, 最差 1.344 ms (平台相关的软指标, 随负载浮动)

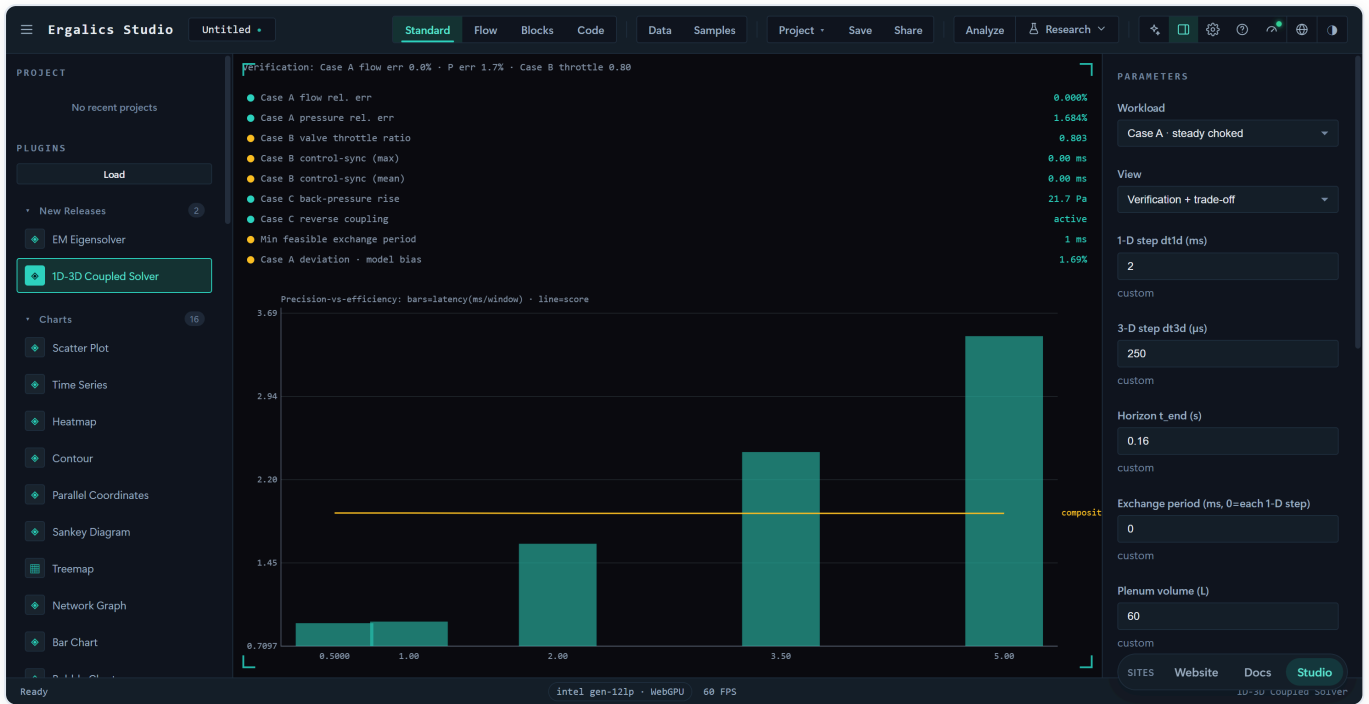
下面集中给出 Case A 的三张视图：3D 场体素、耦合时间序列与精度—效率权衡曲线。



Case A 的 3D 场体素：定常堵塞下 1D 管网经 x0 面把出流注入 3D 场（正向耦合），羽流沿 +x 向出流面发展；出流面再把平均背压回灌给管网（反向耦合）。



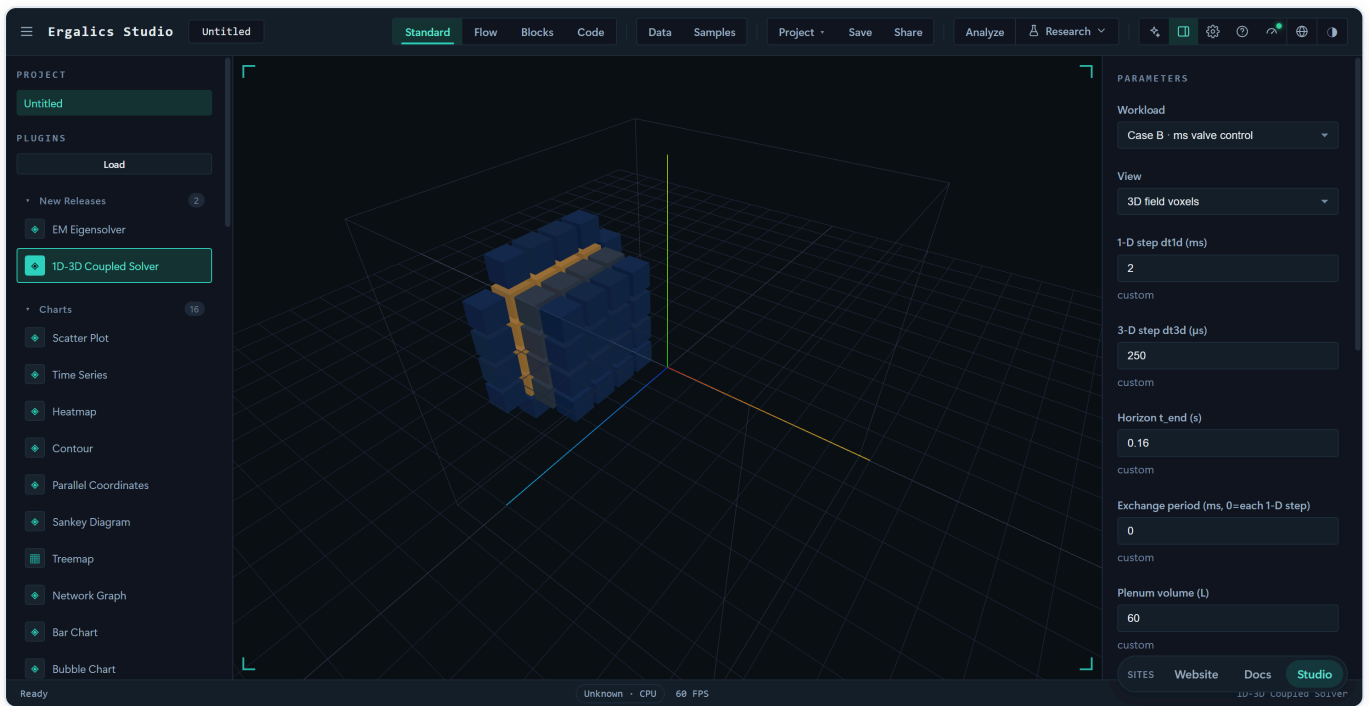
Case A 的耦合时间序列：上部 1D 出口流量叠阀门开度保持稳定，下部 3D 出流面背压随时间平缓演化并叠逐窗口接口误差，用于核对稳定状态下的守恒与响应。



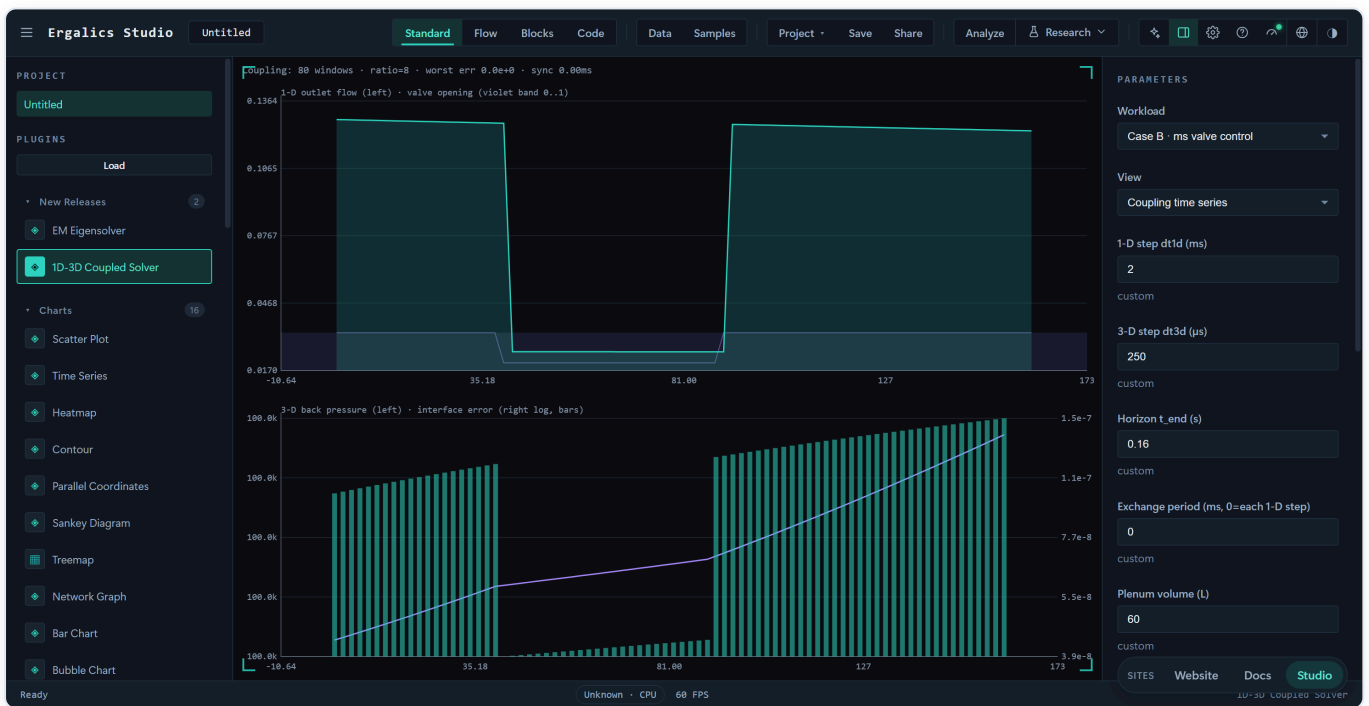
Case A 的权衡曲线：以交换周期为扫描变量，接口误差（柱）叠延迟（柱）与综合评分（连线）同时呈现并标出最优交换周期，在精度与计算效率之间取平衡。

9.2 Case B 毫秒级阀门控制（实测）

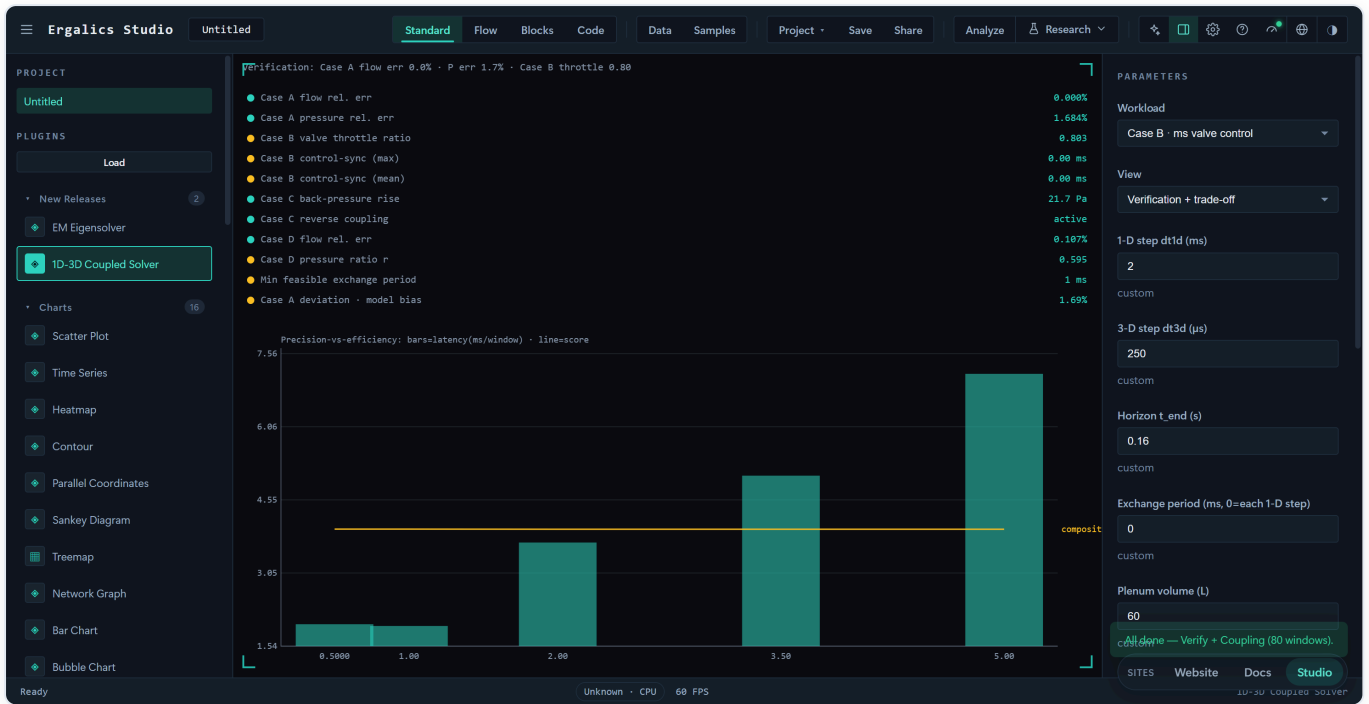
指标	值
事件	t=0 全开 → 40 ms 阶跃 20% → 90 ms 重开 100%
全开流量 / 关闭流量 / 重开流量	0.1281 / 0.02529 / 0.1260 kg/s
节流比 (open-closed)/open	0.803
控制同步误差 max / mean	0 / 0 ms (事件落 2 ms 网格线上)
窗口 / 时长	80 窗口, 0.16 s
终态背压 / 气室压	100.0 kPa / 576.2 kPa



Case B 的 3D 场体素：毫秒级阀门阶跃下出流注入在 +x 向场域内的空间分布，展示正向耦合的质量/焓源项如何改变场内的热含量格局。



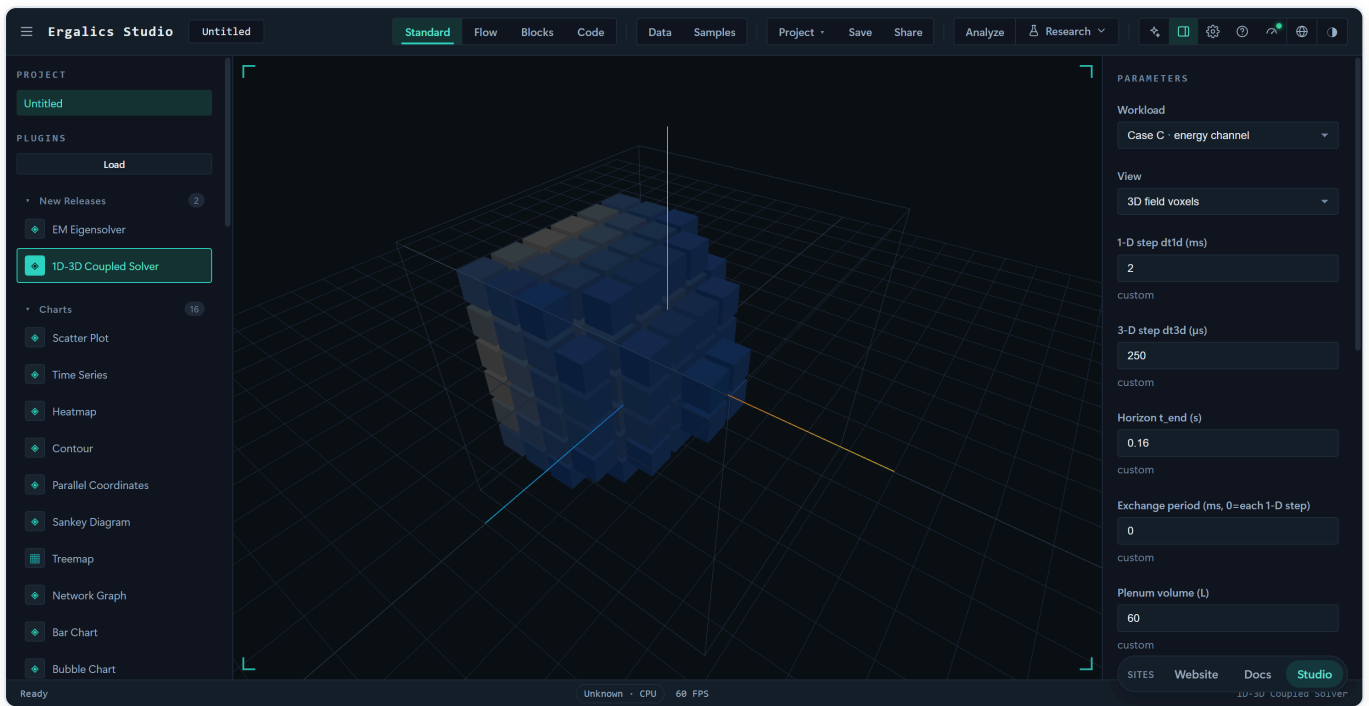
Case B 的耦合时间序列：阀位阶跃（40 ms 关到 20%、90 ms 重开 100%）在 1D 出口流量上形成清晰方波，在 3D 背压上留下对应的阶跃响应，控制同步误差为 0。



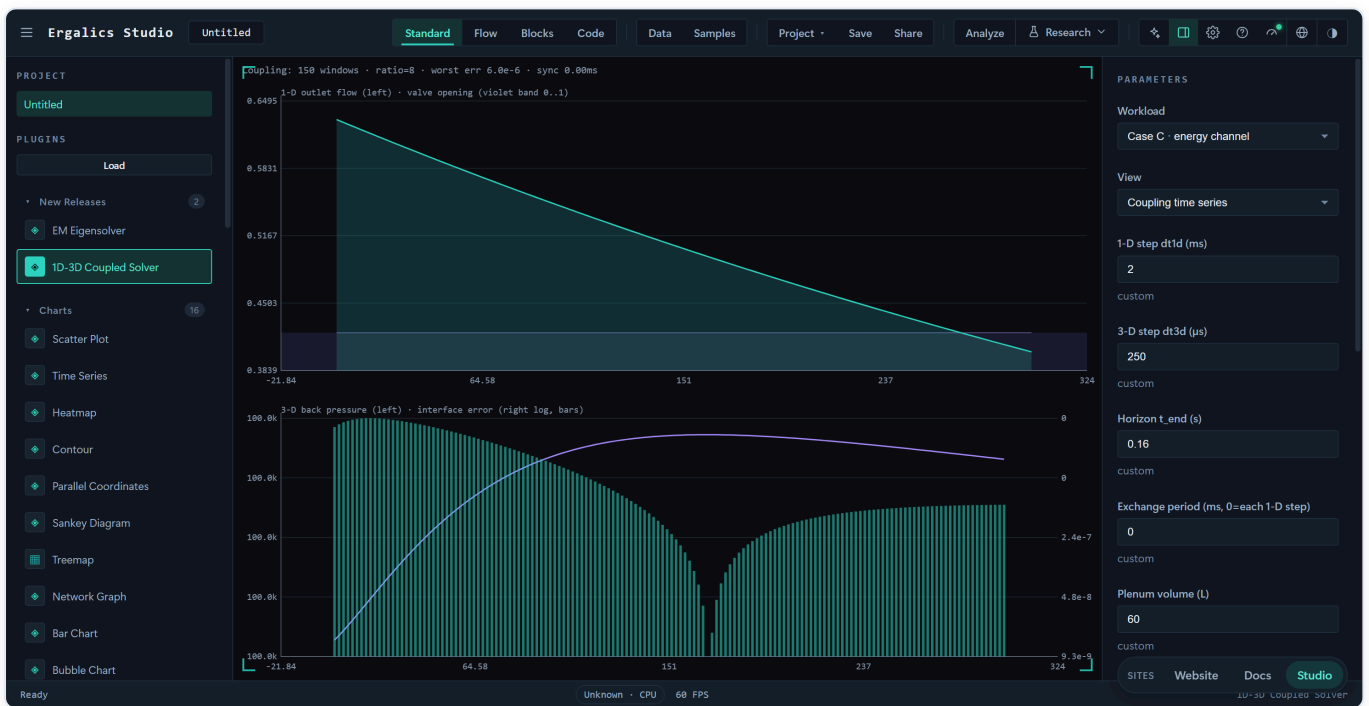
Case B 的权衡曲线：毫秒级阀门控制这一快速瞬态工况下，交换周期扫描的接口误差与延迟此消彼长，提示此处需要足够频繁的交换以跟踪方波阶跃。

9.3 Case C 高扩散能量通道（实测）

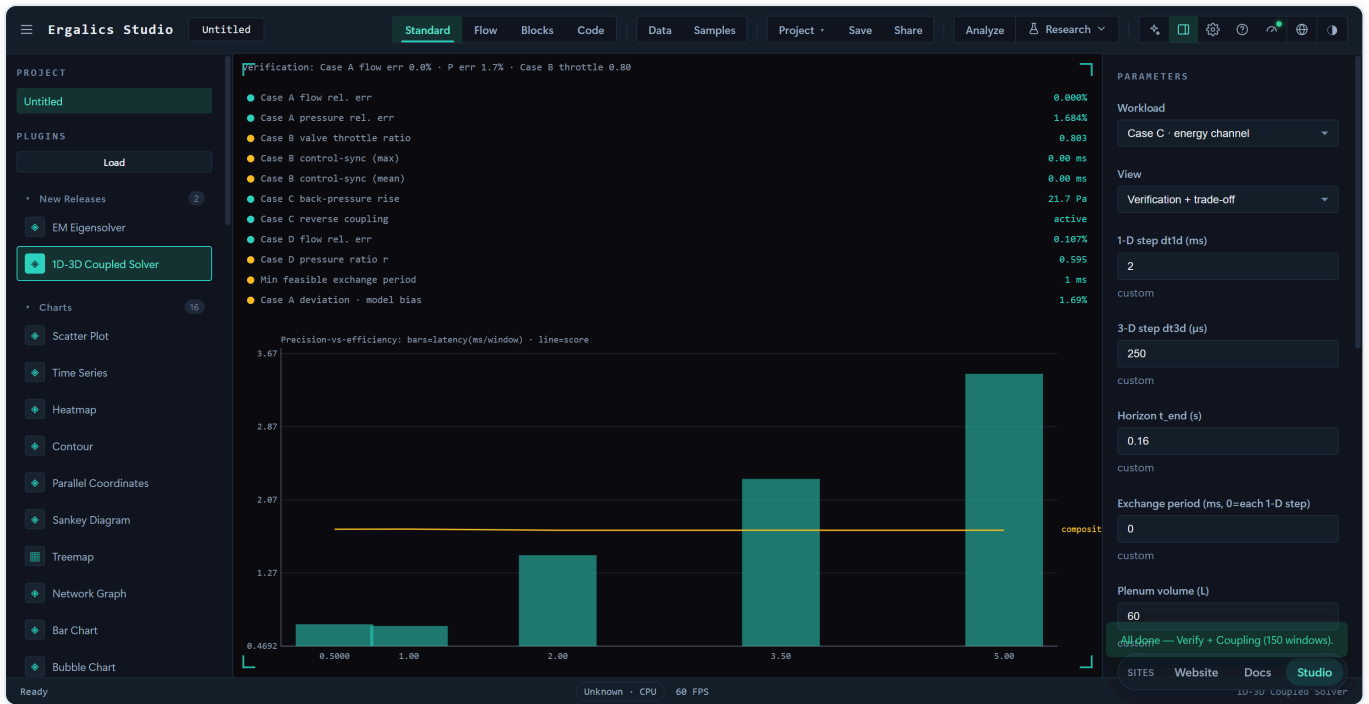
指标	值
出流解析流量	0.6311 kg/s（求解同值，相对误差 0.0% ）
出流面背压抬升	+21.7 Pa （反向耦合真实参与，engagement = active）
窗口 / 时长	150 窗口，0.3 s
接口误差	均值 2e-6，最差 6e-6
终态背压 / 气室压	100.02 kPa / 762.4 kPa



Case C 的 3D 场体素：高扩散系数能量通道下注入热含量在更大范围内铺展，出流面平均背压随之抬升 21.7 Pa，直观呈现反向耦合的真实参与。



Case C 的耦合时间序列：出流面背压随能量注入逐步抬升（engagement = active），反向耦合分支在该工况下被实质驱动。



Case C 的权衡曲线：高扩散能量通道下能量注入较强、背压反馈更显著，曲线提示需要足够频繁的交流以跟踪背压抬升。

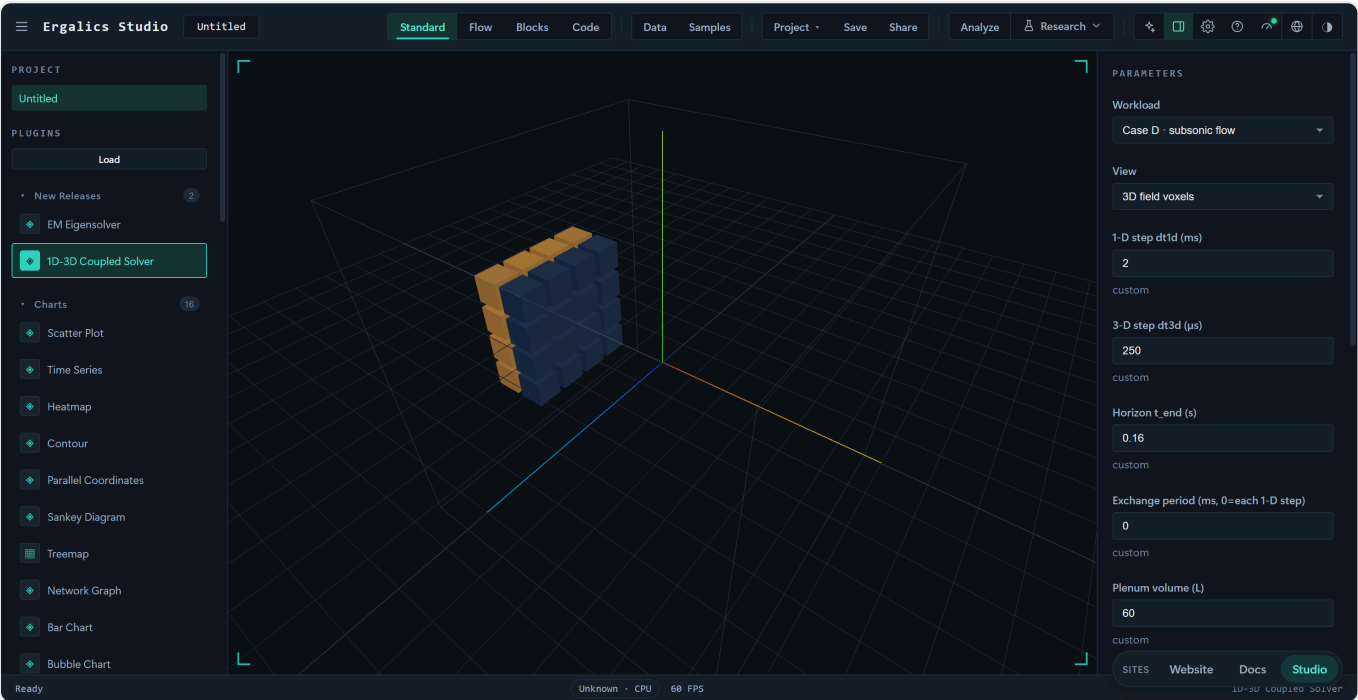
9.4 Case D 亚临界非壅塞流（文献基准对照）

Case A/C 验证的是**壅塞**（choked）分支；Case D 补上赛题"至少两个典型双向耦合用例对照解析/实验基线"（CFD-05）的**非壅塞**侧：压比 $r = p_{back}/p_0$ 被压到临界压比 $PR^* = 0.5283$ 与 1 之间，喷嘴工作在**亚临界**区间，此时出口流量对背压**强敏感**（与壅塞区相反），反向耦合分支在最需要它的工况下被实质性地驱动。出口流量与独立文献中的等熵亚临界收敛喷嘴关系对照：

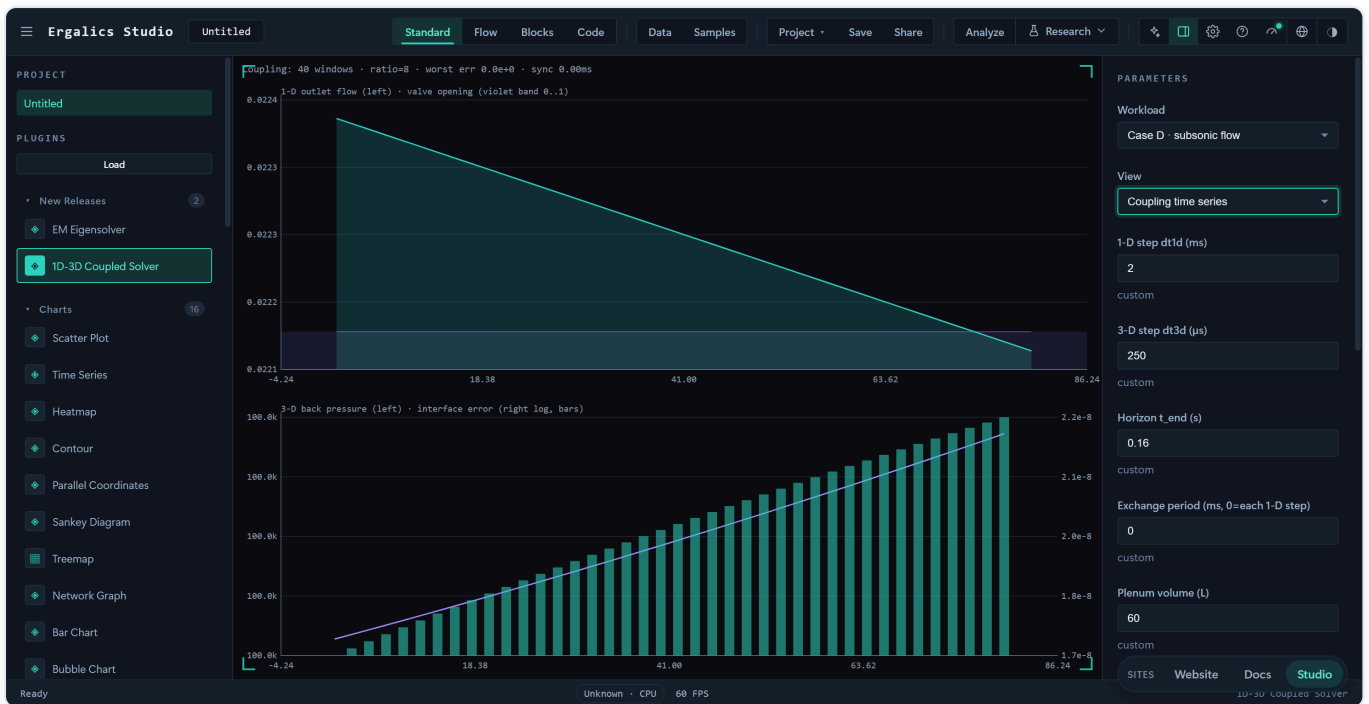
指标	数值	判定
压比 $r = p_{back}/p_0$	0.5954	处于 $(PR^*=0.5283, 1)$ 亚临界区间 ✓
流量（求解器耦合）	0.022113 kg/s	—
流量（文献等熵亚临界）	0.022089 kg/s	相对误差 0.107%
反向耦合参与	active	3-D 背压波幅 0.08 Pa，仍驱动亚临界流量
亚临界区间扫点 <code>subsonic_curve</code>	9 点 / 最大误差 0.0%	求解器复现文献公式到浮点精度

指标	数值	判定
背压敏感度 <code>d ln ṁ/d ln r</code>	-1.83	背压↑ → 流量↓（反向耦合方向正确）
认证断言 <code>certification.all_pass</code>	True	流量误差 ≤ 5% 且亚临界成立

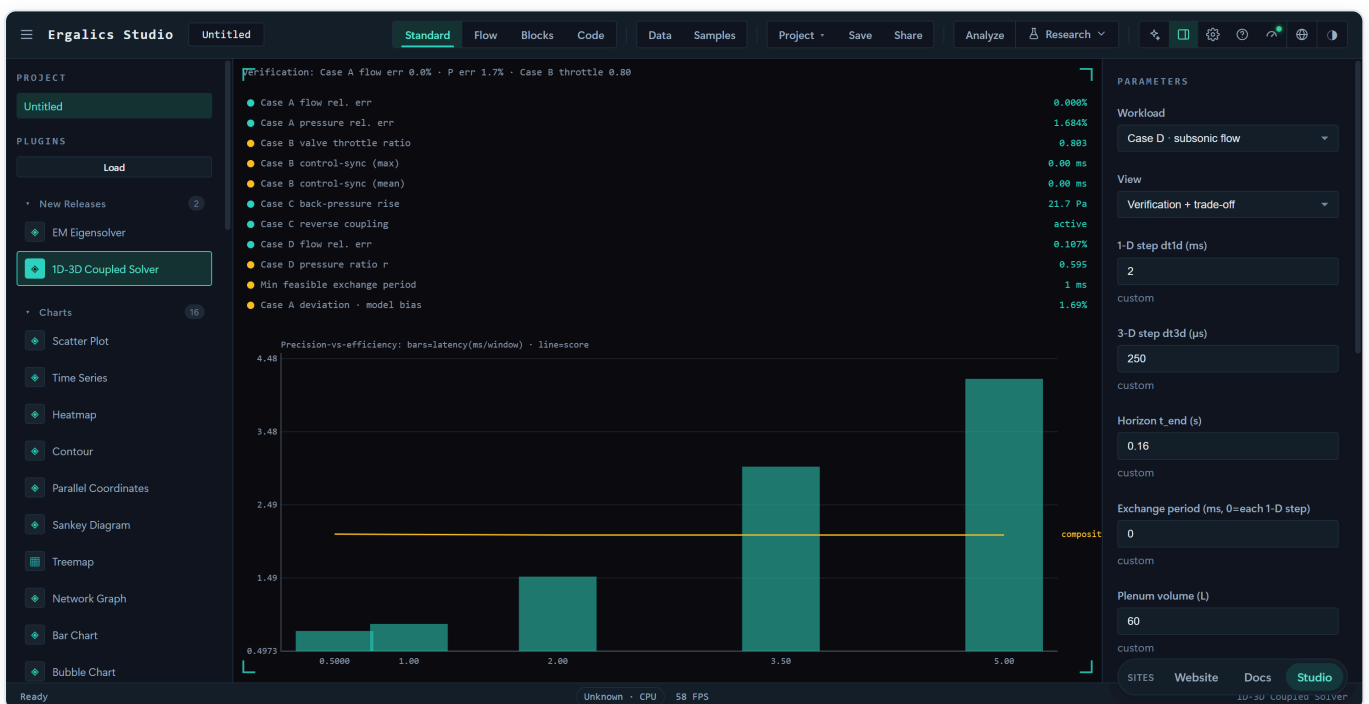
亚临界流量与文献关系精确吻合（0.107%），`verify_subsonic_curve()` 在整段亚临界区间把求解器自身分支与文献等熵公式逐点对照，最大相对误差 0.0%——证明 1-D 喷嘴模型在壅塞/亚临界两个分支上都复现了教科书基准。负背压敏感度 -1.83 坐实"增大 3-D 反馈背压会降低出口流量"这一反向耦合在亚临界区的预期行为。



Case D 的 3D 场体素：亚临界非壅塞区间喷嘴流量对背压强敏感，出流注入在 3D 场内的分布与背压反馈共同决定台中年演进，是反向耦合在最需要它时被驱动的体验。



Case D 的耦合时间序列：3-D 背压波幅约 0.08 Pa、仍持续驱动亚临界出口流量，出口流量与文献等熵亚临界关系在 0.107% 误差内吻合，直观印证“背压↑ → 流量↓”。



Case D 的权衡曲线：亚临界区流量对背压强敏感，交换周期越短、背压反馈越快、接口误差越小。

9.5 权衡曲线与最小周期（实测）

交换周期 (ms)	频率 (Hz)	延迟 (ms)	接口误差	综合评分
0.5	2000	0.378	0	99.886
1.0	1000	0.391	0	99.882
2.0	500	0.705	0	99.788
3.5	285.7	1.246	1e-6	99.699
5.0	200	1.703	1e-6	99.698

最小可行交换周期：预算 1.0 ms、接口容差 0 → `min_feasible_exchange_period = 1.0 ms`（即紧耦合极限 `dt1d`）；请求 0.25/0.5 ms 被钳制到有效 1.0 ms，全程接口误差为 0。上表以 Case A 的扫描为例展示权衡；Case B / C / D 各自的权衡曲线已在 9.2 / 9.3 / 9.4 节按工况分别给出，曲线形态类似、最优周期都落在 1.0 ms 附近——说明"精度—效率权衡"在不同算例间是同一机制在起作用。

9.6 误差归属扫描（实测）

扫描	参数	压力偏差 <code>deviation</code>
喷管 (<code>discharge_coeff</code>)	0.90	0.0154
	0.96	0.0165
	0.98	0.0168
	1.00	0.0172
地平线 (<code>horizon_s</code>)	0.06	0.0081
	0.09	0.0124
	0.12	0.0168
	0.16	0.0230

偏差在喷管参数上几乎不变、随地平线单调增长——与"等熵幂律基线 vs 定温刚性罐模型的累积模型偏差"完全吻合；分解为数值误差 `1.3e-5`、反向耦合贡献 `0.0`、模型偏差 `0.0169`。欢迎把 Case A 的少量压力偏差正确解读为**已文档化的模型选择**，而非数值误差或耦合伪影。

9.7 复现

```
# 本地 CPython (同一求解核心)

cd src/plugins/builtin/fluid-cfd-coupler/python

python -m pytest 2>/dev/null || python run_tests.py          # 运行测试套件

python benchmarks/bench_coupling.py --out ../../../../bench/fluid-cfd-results.json

# 浏览器：插件面板选择算例并点"运行全部"；CLI 与浏览器路径逐字节一致
```

+ 故障场景与处置建议

现象	原因	处置建议
节流比失真	Case B 缺少 t=0 全开事件时， <code>valve_opening</code> 取首个事件开度致"全开"采样段实为节流段	事件表务必先声明 (<code>0.000</code> , <code>1.00</code> , <code>0.0</code>) 基线 (已内置)
浏览器报 <code>'str' object has no attribute 'get'</code>	Worker 桥把 JSON 字符串直接当 dict 传入 <code>solve_json</code>	桥层统一先 <code>json.loads</code> 再调用
验证视图与时间序列"刷出来一样"	视图渲染被数据有无隐式劫持、View 下拉失效	视图改为显式切换，运行动作自动跳到对应视图
点终止仍显示"计算中"	宿主全局状态未复位	终止时同步 <code>setStatus('ready')</code> 复位宿主全局状态，并清空在途任务
3D 场景未挂载	Pyodide 在 Worker 而 3D 需要主线程宿主场景	回退 2D 并提示，重置插件后重跑
3D 体素框比例异常	自定义网格 <code>nx#ny#nz</code> 时等立方体素撑长	已按各轴维度处理，框自适应而非强制立方

十一 测试与可复现性

Python 侧测试覆盖：Case A 匹配解析解与认证、Case B 节流比为正、Case C 能量通道与反向耦合参与、Case D 亚临界文献基线认证、亚临界区间复现文献公式、守恒审计有限、多速率子循环、1D 进推守恒、壅塞闭式解、权衡曲线返回、最小可行交换周期与非平凡阶跃降流。TS 侧 typecheck (`tsc --noEmit`) 通过，渲染与 Figure Studio 面板构造为可单测的纯函数。基准 JSON 由同一求解路径产出，同版本源码重跑可复现第 9.1–9.6 节数字。

为保证可复现性，本插件配一枚独立的一键复现脚本，把“环境 → 测试 → 再生基准 → 核对”串成单条命令，供评审在任意机器上重放第 9.1–9.6 节结论。

11.1 前置条件

依赖	用途	缺失时
Python 3.10+	运行数值内核	不可运行
numpy	数值内核必需	不可运行
scipy	稀疏求解（可选）	自动回退纯 NumPy 后端，结果一致，仅性能略降

11.2 一键复现

```
# 仅验证本赛题
node scripts/repro-plugins.mjs --only=fluid

# 快速模式（本赛题无重型算例，与完整模式步骤一致）
node scripts/repro-plugins.mjs --only=fluid --fast
```

脚本对插件按序执行：`env`（定位可用 python、读取 numpy / scipy 版本）→ `tests`（运行 `python/run_tests.py` 独立断言套件）→ `gen`（经同一入口 `driver.py` 在临时目录重新生成基准 JSON）→ `verify`（逐字段对比新 JSON 与提交基准 `bench/fluid-cfd-results.json`）。退出码 `0` = 测试通过且确定性科学字段一致；`1` = 测试失败或存在确定性科学字段漂移。

手动复现亦可分步执行：

```
cd src/plugins/builtin/fluid-cfd-coupler/python

python -m fluid_cfd.driver verify          # Case A/B/C/D + 亚临界曲线 + 权衡 + 敏感性
python run_tests.py                      # 期望 24 passed, 0 failed
python benchmarks/bench_coupling.py      # 生成基准 JSON
```

注意：`bench_coupling.py` 会直接写入 `bench/fluid-cfd-results.json`（即脚本用于比对的基准文件）。仅当刻意为本赛题更新基准时执行，否则用脚本的临时目录重生成，避免覆盖提交态。

11.3 判定口径

脚本内置确定性字段比较器，把基准 JSON 字段按四类判定：

类别	判定方式	示例
精确断言	字符串 / 布尔必须完全相等	<code>ok</code> 、 <code>subsonic_engaged</code> 、 <code>converged</code>
确定性数值	相对误差 $\leq 1e-6$	<code>flow_rel_error</code> 、 <code>pressure_rel_error</code> 、 <code>valve_throttle_ratio</code> 、 <code>min_feasible_exchange_period_ms</code>
机器相关	忽略或软容差（note 不判 FAIL）	<code>seconds</code> 、 <code>exchange_latency</code> 、 <code>composite_score</code> （内含墙钟延迟等机器量）

- `composite_score` 属机器相关：`trade_off` 综合评分将交换延迟以固定权重混入，延迟为墙钟量，跨机器漂移属预期，判 note。
- 长度不一的数组（如各 case 的 `windows`）按下标对齐逐项比较；元素带 `case` 名的按 `case@method` 复合身份对齐，避免与同 param 异配置的条目错配。
- 软项（note）不计入 FAIL；仅精确断言与确定性数值违背机组失败。

11.4 输出解读

`--only=fluid` 成功输出示例：

```

=== fluid-cfd-coupler (CFD) reproduction: env → tests → regenerate → verify ===
python 3.10.11 | numpy 2.2.6 | scipy 1.15.3

[fluid-cfd-coupler]
  ok   fluid-cfd-coupler: standalone test suite passed
  ok   fluid-cfd-coupler: fresh bench matches committed baseline on all deterministic scientific field
  note fluid-cfd-coupler: ...trade_off[0].composite_score rel=0.00% (latency-blended, within tolerance)

=====
RESULT: PASS – fluid-cfd-coupler reproduces and match committed baselines
=====

```

- `ok ... matches committed baseline`：新算结果与提交基准在全部确定性科学字段上一致。
- `note ... (latency-blended)`：机器相关量漂移，符合预期，不计失败。
- `FAIL` 行按以下顺序排查：① 测试套件是否通过；② 基准文件是否存在；③ 重生成是否报错；④ 哪些科学字段偏离（脚本打印字段路径与新旧值）。

11.5 故障排查

现象	处置
python discovery 失败	安装 Python 3.10+，或设环境变量 <code>PYTHON=/path/to/python</code>
numpy <code><none></code>	<code>pip install numpy</code>
scipy <code><none></code>	可选；内核自动用纯 NumPy 后端，结果一致
某科学字段 FAIL	按 11.4 节顺序排查

11.6 交付物对照

交付项	位置
一键复现脚本	<code>scripts/repro-plugins.mjs</code> (<code>--only=fluid</code>)
数值内核 + 验证	<code>src/plugins/builtin/fluid-cfd-coupler/python/</code> (24 项测试)
解题说明书	<code>src/plugins/builtin/fluid-cfd-coupler/SOLUTION.md</code>
复现基准	<code>bench/fluid-cfd-results.json</code>

十二 安全与合规设计

- **无上传、无网络**：求解全程本地，无闭源/云端核心能力。
- **导出仅本地文件**：Figure Studio、诊断报告、基准 JSON 均为本地写盘。
- **沙箱 trusted**：插件声明 `sandbox: 'trusted'`，运行于宿主 Worker，无额外敏感权限。
- **负载护栏**：3D 场尺寸、窗口数在 `normalized()` 校验，杜绝意外的大内存分配。

十三 与赛题要求的对应

赛题要求	本方案对应
双典型工况对照解析/实验基线	Case A（定常堵塞，流量 0.0% 误差）+ Case B（毫秒级阀门控制）+ Case C（能量通道）+ Case D（亚临界文献），见第九节
时间步长协调	多速率子循环 <code>dt1d/dt3d</code> ，第二节；约束在 <code>normalized()</code> 校验
双向边界耦合	正向质量/焓注入 + 反向出口平均背压回灌，第三节
守恒性与界面误差	<code>iface_error</code> 审计 + <code>mean/worst_interface_error</code> ，见 3.3
毫秒级控制逻辑及其同步误差	阀门事件表（阶跃/斜坡）+ <code>control_sync_max/mean_ms</code> ，第四节
交换频率与精度-效率权衡	<code>trade_off()</code> + <code>min_feasible_exchange_period()</code> + <code>composite_score</code> ，第六节、9.5 节
误差归因与可信度	误差归属分解（数值/耦合/模型偏差）+ 逐项认证，见 6.3、9.6 节

赛题要求	本方案对应
方案说明文档	本文档（原理、复杂度、基准、故障、安全）
可运行原型	浏览器插件（四种视图）+ 本地 <code>run_tests.py</code> / <code>bench_coupling.py</code> 共用同一核心
可复现材料	源码、 <code>requirements.txt</code> 、 <code>fluid_cfd/README.md</code> 、 <code>bench/fluid-cfd-results.json</code> 、 <code>run_tests.py</code>

十四 已知边界与后续工作

项	现状	影响与建议方向
3D 物理保真	单标量（热含量）扩散-平流，非 NS	若需真实流场可接入 NS/低马赫近似，但会显著抬高 Pyodide 成本
1D 网络拓扑	单气室 + 单喷管	管网化（串并联、摩擦、传热）后需复用同一耦合机
分数时间比	要求 <code>dt3d</code> 整除 <code>dt1d</code>	有意的简化以保证窗口对齐；分数比需重写子循环边界
交换频率自适应	交换周期为定值	可结合第 9.5 节权衡与最小周期实现动态换窗
三维渲染细节	体素粗细固定	可加入 isosurface/剖切面、阈值滑杆
前端端到端覆盖	仅纯逻辑单测 + typecheck	建议补一条"点示例并断言收敛状态"的 e2e 用例

让每一次科学计算，

都发生在离思想最近的地方

Ergalics Studio 把复杂度收进浏览器：安装、配置与数据搬运都不该出现在科研的路上。打开即可计算，数据留在本机，四种工作模式共享同一份数据——一张图、一条流程与一行代码，都是同一次工作的不同表达。

工具越安静，思想就越清楚。我们做的每一处取舍，都为了把注意力还给问题本身。

作者 [SnowLeopard-io](#) github.com/SnowLeopard-io

官网 snowleopard-io.github.io/ErgalicsStudio

GitHub github.com/SnowLeopard-io/ErgalicsStudio

Gitee gitee.com/cnt-code/ergalics-studio

独立专题文档 · Standalone Note